REGISTRO DE DECISÃO DE ARQUITETURA

ADR-001: Modernização e Evolução da Plataforma de Gestão de Fluxo de Caixa

Data: 10 de outubro de 2026

Status: Proposto

Autoria: Alessandra Santos

Domínio de Negócio: Gestão Financeira (Cash Flow Management)

SEÇÃO 1: CONTEXTO - O CENÁRIO ATUAL (AS-IS)

1.1 Descrição do Problema e Cenário Legado

A plataforma atual de Gestão de Fluxo de Caixa opera sob uma arquitetura monolítica implementada num servidor *on-premise* (servidor físico). Todo o fluxo de negócio, desde a verificação de acessos até à gravação de vendas e cálculo de relatórios, ocorre num processo partilhado e centralizado, o que gera graves problemas de concorrência por recursos computacionais.

Ao observar a topologia de acesso à interface, verifica-se que o comerciante comunica com a aplicação através de uma ligação direta via *HTTP* (*Hypertext Transfer Protocol* / Protocolo de Transferência de Hipertexto), utilizando apenas uma autenticação básica de sessão com credenciais de acesso (*login* e senha). A arquitetura atual carece totalmente de componentes de proteção de fronteira (borda): não existe um *API Gateway* (Portão de Entrada de Aplicações) para orquestrar pedidos, não há um *WAF* (*Web Application Firewall* / *Firewall* de Aplicações *Web*) para mitigar ataques maliciosos, e a ausência de *Rate Limiting* (Limitação de Taxa de Pedidos) deixa a infraestrutura vulnerável a sobrecargas. A Figura 1 ilustra detalhadamente esta topologia de acesso direto e a dependência de um servidor único.

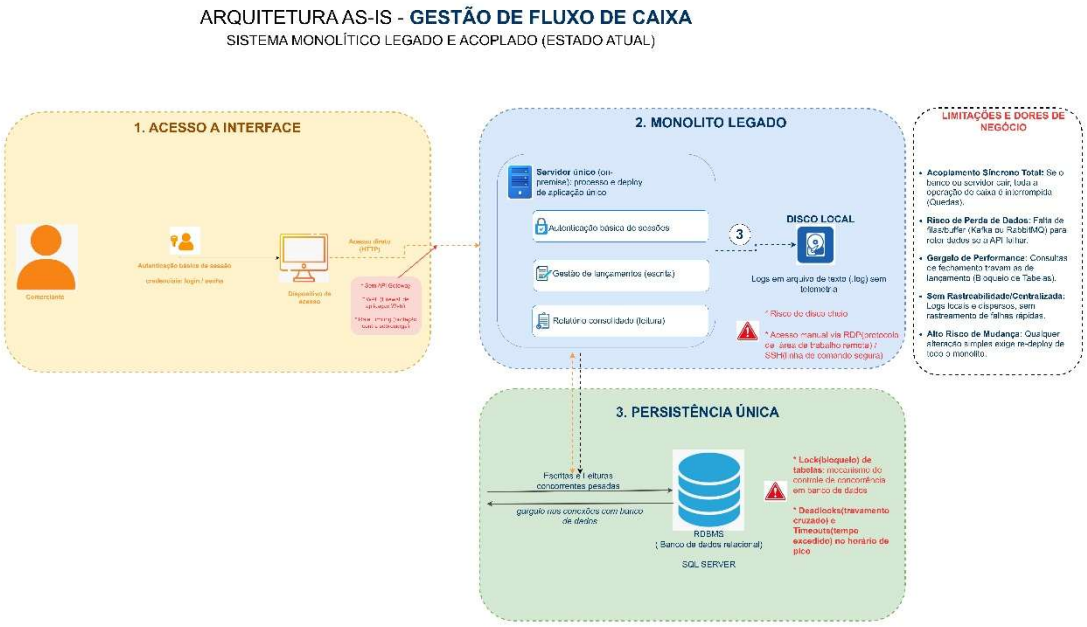


Figura 1 – Arquitetura AS-IS – Gestão de Fluxo de Caixa

Esta estrutura de persistência única (uma única base de dados para todo o sistema) e processos fortemente ligados resulta nas seguintes limitações críticas de negócio e operação:

- **Acoplamento Síncrono Total:** Existe uma interdependência direta entre todos os módulos. Se o servidor sofrer uma quebra (queda) ou a rotina de consolidação da base de dados falhar, toda a operação da frente de caixa é imediatamente interrompida.
- **Gargalo de Performance (Desempenho) e Persistência Única:** O sistema utiliza um único *RDBMS* (*Relational Database Management System* / Sistema de Gestão de Bases de Dados Relacionais), especificamente o *SQL Server*. Nos horários de pico, as escritas das vendas concorrem diretamente com as leituras pesadas dos relatórios. Isto gera *Locks* (bloqueios de tabelas de dados), *Deadlocks* (travamentos cruzados onde dois processos se bloqueiam mutuamente) e consequentes *Timeouts* (esgotamento do tempo limite de resposta).
- **Risco de Perda de Dados:** A arquitetura legada não possui qualquer mecanismo de tolerância a falhas, por exemplo, *brokers* de mensageria. Se a base de dados ficar indisponível no exato momento da venda, os dados da transação perdem-se de forma irrecuperável.
- **Falta de Rastreabilidade Centralizada:** Os registros de operação e erros (*Logs*) são guardados em ficheiros de texto simples (.log) num disco rígido local, sem qualquer mecanismo de telemetria ou monitorização proativa. Existe um risco constante de paragem do servidor devido a disco cheio. Adicionalmente, para analisar uma falha, a equipe técnica necessita acessar manualmente ao servidor através de *RDP* (*Remote Desktop Protocol* / Protocolo de Ambiente de Trabalho Remoto) ou *SSH* (*Secure Shell* / Terminal Seguro de Linha de Comandos).

- **Alto Risco de Mudança:** Qualquer alteração simples num ficheiro ou correção de um pequeno erro exige o *re-deploy* (reimplantação) integral de todo o monólito, o que acarreta um risco elevado de paralisar toda a operação do sistema financeiro durante as atualizações.

Compreendidas as limitações físicas e os gargalos de infraestrutura do cenário legado, torna-se imperativo analisar a arquitetura lógica que origina estas falhas. A estrutura do código e das regras de negócio reflete a mesma rigidez do servidor único, carecendo de fronteiras claras entre os diferentes contextos da operação.

1.2 Mapeamento de Domínios Funcionais (Visão AS-IS)

No estado legado, o conceito de separação de domínios não existe tecnicamente, de forma que todas as regras de negócio residem no mesmo espaço de memória e competem pelos mesmos recursos.

Para efeitos de diagnóstico arquitetural, o domínio principal (Gestão Financeira) está atualmente acoplado da seguinte forma:

1. **Subdomínio de Acessos (Altamente Acoplado):** Engloba o módulo de autenticação básica. Não possui base de dados própria de utilizadores; as credenciais são validadas no mesmo processo que regista o dinheiro do caixa.
2. **Subdomínio de Frente de Caixa (Altamente Acoplado):** O módulo de Gestão de Lançamentos, cuja responsabilidade crítica é a escrita (registro) de débitos e créditos. Depende intrinsecamente do estado de saúde global do servidor.
3. **Subdomínio de Consolidado Gerencial (Altamente Acoplado):** O módulo de Relatório Consolidado, responsável pela leitura dos dados. A sua execução degrada diretamente o desempenho do subdomínio de frente de caixa, visto que ambos leem e escrevem nas mesmas tabelas físicas.

Este elevado nível de acoplamento e a consequente ausência de fronteiras lógicas não configuram apenas um débito técnico de engenharia; traduzem-se num impacto direto e mensurável na operação diária do comerciante. Para clarificar esta relação de causa e efeito, o mapeamento a seguir detalha como a estrutura monolítica compromete a execução das capacidades fundamentais do negócio.

1.3 Capacidades de Negócio na Visão AS-IS

A tabela a seguir evidência como as capacidades que o negócio necessita para operar de forma saudável estão severamente limitadas pela topologia tecnológica atual:

Capacidade de Negócio	Módulo Tecnológico Legado	Limitação Atual na Execução
C1. Acesso ao Sistema e Segurança	Autenticação Básica (Sessão / Monólito)	**Insegurança e Fragilidade:** O comerciante acessa sem encriptação avançada ou proteção contra sobrecargas. Se ocorrerem acessos simultâneos massivos, o sistema colapsa antes mesmo do *login*.

C2. Registro Diário de Vendas	Gestão de Lançamentos (Escrita Síncrona)	**Falta de Fiabilidade:** A capacidade mais crítica do lojista concorre por recursos. Uma quebra pontual no banco relacional impede o registro da venda, causando paragem na loja e perdas financeiras.
C3. Visualização do Saldo Consolidado	Relatório Consolidado (Leitura Síncrona)	**Degradação de Desempenho:** A consulta de saldos exige operações matemáticas pesadas que bloqueiam tabelas. Puxar um relatório trava as caixas registadoras em pleno horário de pico.
C4. Auditoria e Resolução de Problemas	Ficheiros Locais e Acesso *RDP/SSH*	**Opacidade Operacional:** Falhas exigem intervenção manual reativa da equipe de engenharia, que despende horas a analisar ficheiros de texto não estruturados, resultando em longos tempos de inatividade.

1.4 Representação Arquitetural (C4 Model) - Visão AS-IS

Para consolidar o diagnóstico das vulnerabilidades e do acoplamento sistêmico descritos nas seções anteriores, a arquitetura legada foi mapeada utilizando o padrão *C4 Model*. As visões abaixo ilustram o acesso direto, a implantação num servidor único e o gargalo interno de componentes a concorrerem pela mesma base de dados.

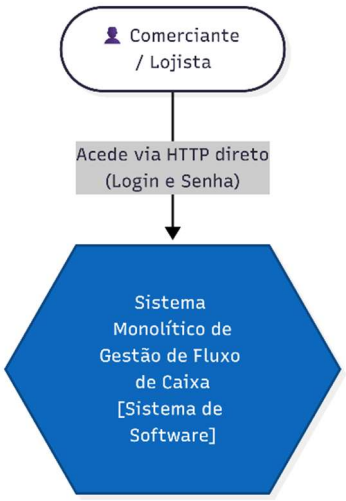


Figura 2 – Nível 1 – Diagrama de Contexto (visão C4 Model)

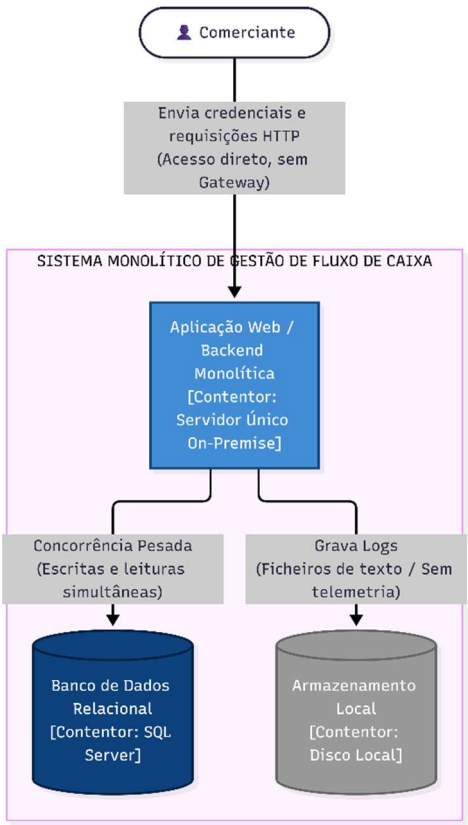


Figura 3 – Nível 2 – Diagrama de Container (visão C4 Model)

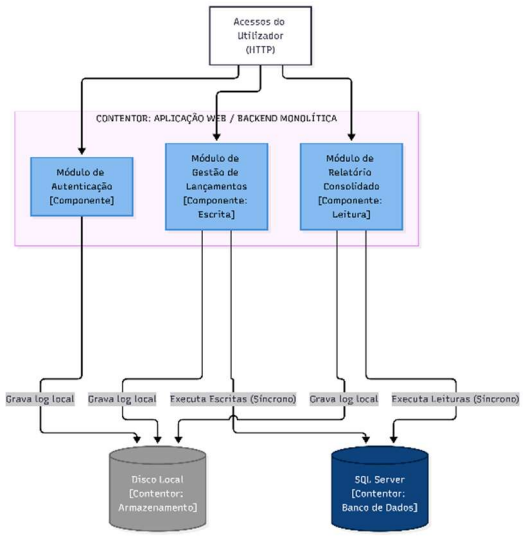


Figura 4 – Nível 3 – Diagrama de componentes (visão C4 Model)

Como evidenciado na Visão de Componentes (figura 4), a restrição crítica da arquitetura reside no fato de todos os módulos de negócio compartilharem o mesmo processo computacional e concorrerem de forma síncrona pela mesma base de dados relacional. Este gargalo estrutural comprova que as interrupções operacionais e a degradação de desempenho não podem ser solucionadas exclusivamente através do aumento de capacidade física do servidor (escalabilidade vertical). Portanto, assegurar a resiliência e a escalabilidade da plataforma exige uma mudança estrutural de paradigma. A estratégia

para esta modernização — dividida entre uma arquitetura de transição e uma solução definitiva orientada a serviços de nuvem — será fundamentada e detalhada na Seção 2 (Decisão).

SEÇÃO 2: DECISÃO

Após a análise exaustiva da arquitetura atual, concluiu-se que a modernização integral (substituição total para a nuvem) implicaria riscos operacionais inaceitáveis e retardaria a estabilização do fluxo transacional do negócio. Esta justificativa para rejeitar uma migração imediata de bloco único fundamenta-se em três dimensões arquiteturais críticas:

1. **Riscos Operacionais Inaceitáveis:** Na engenharia de sistemas, substituir um monólito legado por uma infraestrutura distribuída numa única implantação maximiza o risco de impacto. A introdução simultânea de consistência eventual, latência de rede e novos padrões de persistência elevaria o risco de um colapso total. Adicionalmente, em caso de falha crítica em produção, a complexidade de *Rollback* (Reversão) — que exigiria a ressincronização de dados distribuídos de volta para a base relacional antiga sem perda de transações — causaria interrupções severas.

2. **Retardamento da Estabilização (O Custo da Demora / *Time-to-Market*):** A construção do ecossistema definitivo (*Cloud-Native* / Nativo na Nuvem) é um projeto de engenharia de médio a longo prazo. Aguardar a sua conclusão significaria prolongar a degradação atual, onde os *locks* (bloqueios de tabelas) e *deadlocks* (travamentos cruzados) continuam a derrubar o sistema nos horários de pico. Uma intervenção arquitetural intermédia é obrigatória para estabilizar a operação a curto prazo.

3. **Proteção do Fluxo Transacional (O Caminho Crítico e Garantias *ACID*):** O fluxo transacional representa o *Critical Path* (Caminho Crítico) de geração de receita da organização, exigindo o cumprimento estrito das propriedades *ACID* (*Atomicity, Consistency, Isolation, Durability* / Atomicidade, Consistência, Isolamento, Durabilidade). As operações de caixa (débitos e créditos) não podem competir por recursos computacionais (processador e memória) com tarefas analíticas secundárias, como a extração de relatórios consolidados.

Por conseguinte, de forma estratégica, a evolução sistémica foi dividida em duas etapas arquiteturais distintas:

A Visão Tática (Transição) atua como uma solução arquitetural intermédia (*MVP - Minimum Viable Product* / Produto Mínimo Viável). O seu objetivo primário é mitigar imediatamente as interrupções operacionais através do desacoplamento lógico entre os fluxos de escrita (lançamentos) e de leitura (relatórios). O grande benefício desta abordagem incremental é a estabilização da plataforma, estancando os travamentos sistémicos no PDV (Ponto de Venda). Esta fase introduz também a ingestão contínua de dados do sistema legado para uma estrutura paralela, utilizando o padrão *CDC* (*Change Data Capture* / Captura de Dados Alterados), preparando a base de dados para a futura migração para a nuvem.

A Visão Target (Definitiva) justifica-se pela necessidade estratégica de atingir a maturidade máxima em resiliência, segurança e escalabilidade. Ao migrar integralmente para um ecossistema *Cloud-Native* (Nativo na Nuvem) ancorado em serviços geridos (*PaaS - Platform as a Service* / Plataforma como Serviço) e *Serverless* (Computação sem Servidor), a organização elimina o esforço operacional associado à gestão da infraestrutura física. A arquitetura passa a atuar como um habilitador de inovação, suportando integrações avançadas como *IA* (Inteligência Artificial) e conciliação bancária automatizada. O benefício central desta visão é a elasticidade automática (capacidade de escalar recursos computacionais sob demanda) para absorver os picos de tráfego e a implementação do padrão *Zero-Trust* (Confiança Zero) para garantir segurança de nível corporativo.

SEÇÃO 2.1: VISÃO TÁTICA - DESACOPLAMENTO E MENSAGERIA

Para mitigar a degradação imediata de desempenho (*performance*) e preparar a infraestrutura para a futura escalabilidade na nuvem, a Visão Tática implementa o padrão *CQRS* (*Command Query Responsibility Segregation* / Segregação de Responsabilidade de Comandos e Consultas) a nível lógico e de infraestrutura, em conjunto com uma arquitetura *Event-Driven* (Arquitetura Orientada a Eventos). A plataforma foi dividida em domínios isolados, permitindo a escalabilidade independente de cada componente operacional.

A figura 5 apresenta a arquitetura dividida em domínios isolados, permitindo escalabilidade independente.

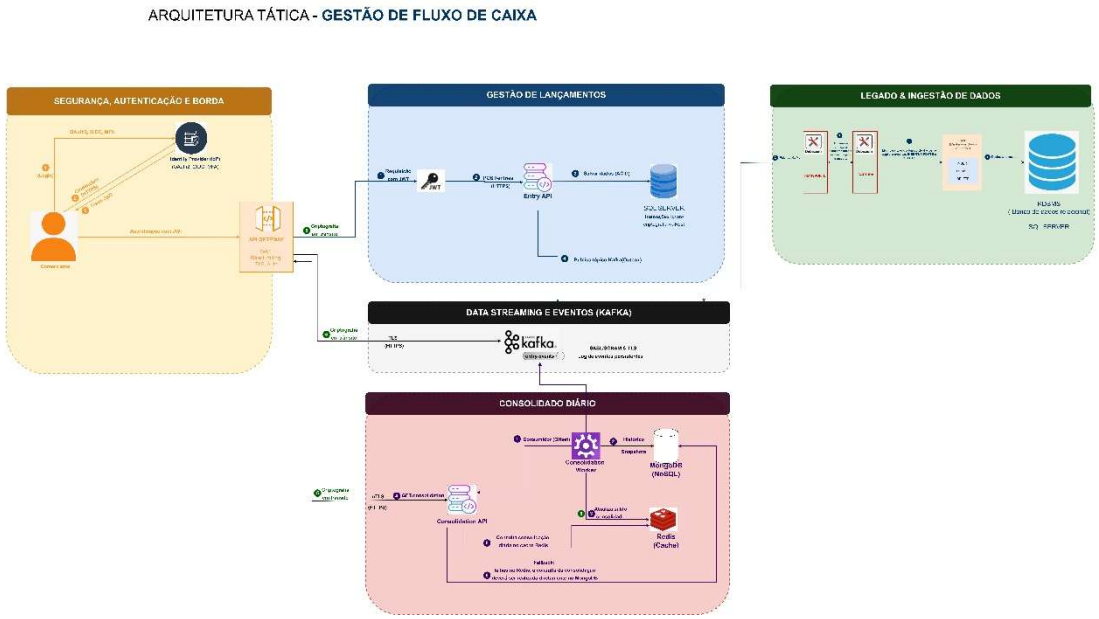


Figura 5 – Arquitetura tática

O fluxo de processamento de um lançamento financeiro até à sua consolidação obedece à seguinte topologia:

1. **Fronteira de Acesso e Autenticação:** O lojista realiza o pedido via *HTTPS* (*Hypertext Transfer Protocol Secure* / Protocolo de Transferência de Hipertexto Seguro). A validação adota a premissa de *Zero-Trust* (Confiança Zero), utilizando um *Identity Provider* (Provedor de Identidade) externo que emite um *Token JWT* (*JSON*

Web Token). A requisição validada atinge o *API Gateway* (Portão de Entrada de Aplicações), que aplica políticas de *Rate Limiting* (Limitação de Taxa de Pedidos para máximo de 50 RPS) e *WAF* (*Web Application Firewall*), roteando o tráfego de forma segura.

2. **Gestão de Lançamentos (Domínio de Escrita / *Command*):** A requisição é processada pela *Entry API* (Interface de Entrada de Dados), um microsserviço autônomo dedicado à inserção transacional. A gravação ocorre numa base de dados relacional (*SQL Server*), garantindo rigorosa conformidade *ACID* (*Atomicity, Consistency, Isolation, Durability* / Atomicidade, Consistência, Isolamento, Durabilidade). O processamento é finalizado em milissegundos, libertando imediatamente o terminal do utilizador.

3. **Ingestão e Barramento de Eventos (*CDC* e Mensageria):** Através do padrão *CDC* (*Change Data Capture* / Captura de Dados Alterados), as alterações físicas na base de dados relacional são capturadas de forma não-intrusiva e transformadas num evento de negócio. O padrão *Transactional Outbox* publica este evento no *Apache Kafka* (um *Broker* / Barramento de Mensagens de alta retenção em disco). O *Kafka* atua como um mecanismo robusto de tolerância a falhas, retendo as mensagens com segurança caso os serviços subsequentes fiquem inoperantes (garantindo perda de eventos inferior a 5%).

4. **Consolidado Diário (Domínio de Leitura / *Query*):** O *Consolidation Worker* (Serviço de Processamento em Segundo Plano) consome as mensagens do *Kafka* de forma assíncrona. O *Worker* calcula o novo saldo e materializa os dados em duas camadas: dados quentes (*Hot Data* / acesso rápido em memória) no *Redis*, e dados frios (*Cold Data* / histórico documental) no *MongoDB* (Base de Dados *NoSQL*). A consulta gerencial é feita através da *Consolidation API*, que lê o saldo diretamente do *Redis* em tempo constante $O(1)$, suportando picos de tráfego sem bloquear as tabelas de inserção de vendas.

O Mapeamento de Valor e Capacidades Técnicas (Visão Tática) visa demonstrar os ganhos imediatos alcançados por esta arquitetura intermédia, apresentando um quadro que correlaciona cada domínio de negócio tático à respetiva capacidade técnica implementada localmente (ou em ambiente IaaS) e evidenciando o valor inicial entregue à operação do comerciante:

Domínio de Negócio e Capacidade	Componentes Técnicos (IaaS/MVP)	Valor Entregue à Operação
Segurança e Borda	API Gateway (com Rate Limiting e WAF) e Provedor de Identidade Externo	**Estabilização de Acesso:** Estanca as sobrecargas na aplicação legada ao limitar os pedidos a 50 RPS e implementa o padrão *Zero-Trust* com JWT, eliminando a autenticação insegura.

Operações de Frente de Caixa	Entry API (Contentor) + SQL Server (ACID)	**Desacoplamento de Gravação:** O lojista passa a gravar vendas numa API autônoma que não concorre com a extração de relatórios, garantindo que o fechamento de caixa seja executado em milissegundos.
Transporte Assíncrono (Eventos)	Padrão CDC + Apache Kafka (*Broker*)	**Tolerância Inicial a Falhas:** Erradica a perda de transações. Se a base de dados de leitura falhar, as vendas ficam retidas com segurança no disco do Kafka (perda < 5%).
Relatórios Gerenciais (Consolidado)	Worker/Consolidation API + MongoDB (Frio) + Redis (Quente)	**Consultas Sincronizadas:** O relatório é extraído de uma memória *cache* independente (Redis). O gerente pode analisar o fluxo financeiro em horário de pico sem travar as caixas registadoras.

2.1.1 Representação Arquitetural (C4 Model) - Visão Tática

Para ilustrar a estratégia intermédia de estrangulamento do monólito e a segregação de domínios, a arquitetura tática foi mapeada utilizando o padrão *C4 Model*. As visões abaixo ilustram o isolamento lógico das APIs e a introdução da mensageria para estabilizar a operação.

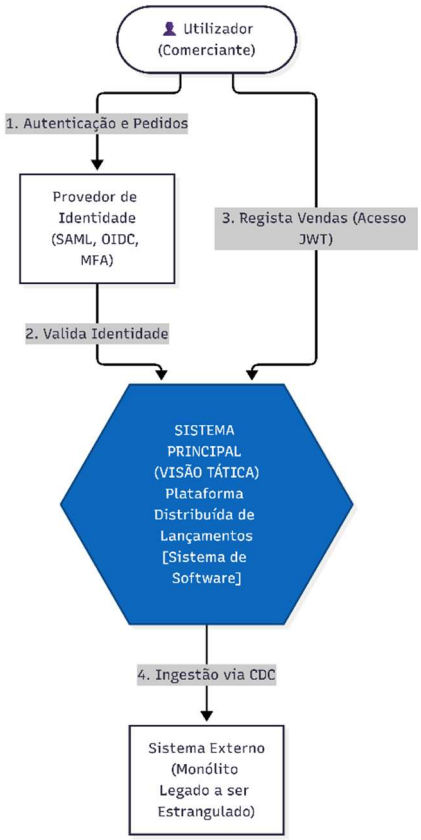


Figura 6 – Diagrama de contexto

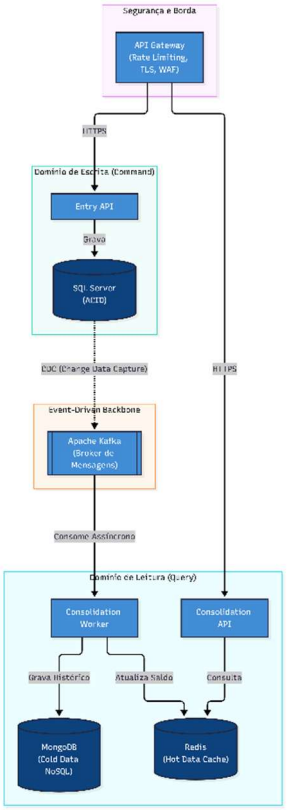


Figura 7 – Diagrama de container

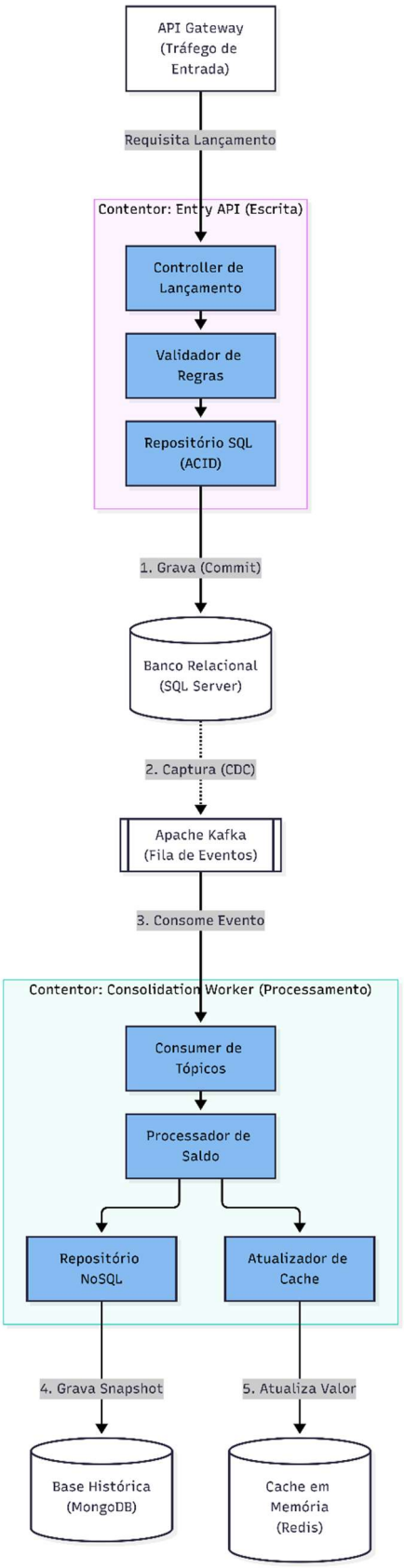


Figura 8 – Diagrama de componentes

2.1.2 Requisitos (Visão Tática)

A partir do fluxo de execução da arquitetura intermédia, os requisitos funcionais, não-funcionais e técnicos são:

Requisitos funcionais:

- **RF-01 (Registro de Movimentação)**: O sistema deve fornecer uma API (Entry API) capaz de receber, validar e registar lançamentos financeiros (débitos e créditos) iniciados pelo lojista.
- **RF-02 (Ingestão Legada - CDC):** O sistema deve capturar de forma assíncrona, através de *Change Data Capture* (CDC), as alterações realizadas na base de dados legada e transformá-las em eventos de negócio.
- **RF-03 (Consolidação de Saldo):** O sistema deve possuir um processo em *background* (*Worker*) que consuma os eventos de lançamentos e processe matematicamente a consolidação do saldo financeiro diário por estabelecimento.
- **RF-04 (Consulta de Relatório):** O sistema deve fornecer uma API (Consolidation API) para que o lojista consulte o saldo consolidado atualizado, de forma independente do fluxo de escrita.

Requisitos não-funcionais:

- **RNF-01 (Desempenho e Throughput):** As APIs devem suportar picos de tráfego de até 50 Requisições Por Segundo (RPS) sem degradação de tempo de resposta. As requisições de consulta de saldo devem ter complexidade temporal O(1) (Tempo constante).
- **RNF-02 (Resiliência e Tolerância a Falhas):** A perda máxima tolerável de eventos/transações é fixada no rigoroso limite de 5%. O sistema de mensageria deve atuar como *buffer* (retenção em disco/fila) para garantir que picos de carga não resultem em perda de dados caso as APIs de leitura estejam indisponíveis.
- **RNF-03 (Isolamento de Domínio / CQRS):** A disponibilidade do domínio de escrita (Gestão de Lançamentos) não pode ser impactada pela indisponibilidade ou lentidão do domínio de leitura (Consolidado Diário), eliminando o "Acoplamento Síncrono Total" do legado.

Requisitos técnicos:

- **RT-01 (Stack de Desenvolvimento):** O código fonte das APIs e do Worker deve ser escrito utilizando a linguagem de domínio do time de engenharia(programação).
- **RT-02 (Bancos de Dados Poliglotas):** O armazenamento deve respeitar a especialidade de cada domínio:
    - *Command:* SQL Server (garantia transacional ACID).
    - *Query (Hot Data):* Redis Cache (para consulta ultrarrápida do saldo).
    - *Query (Cold Data):* MongoDB (NoSQL) para armazenamento de histórico de *snapshots*.

- **RT-03 (Mensageria MVP):** O transporte assíncrono de eventos e o padrão *Transactional Outbox* devem ser implementados sobre o **Apache Kafka** (devido à sua alta retenção em disco comparado ao RabbitMQ).
- **RT-04 (Conteinerização):** A execução e orquestração local (ambiente de desenvolvimento) de toda a infraestrutura e código deverá ser feita exclusivamente via Docker/Docker Compose.

2.1.3 Estimativa de Custos, Dimensionamento e FinOps / FinOps (Visão Tática)

A estimativa de custos para o ambiente de transição (MVP) baseia-se no provisionamento de Máquinas Virtuais (IaaS) operando de forma contínua (730 horas/mês) para suportar o *cluster Docker/Docker Compose*. A metodologia de cálculo considera os seguintes parâmetros de dimensionamento:

- **Entry API, Consolidation API e Worker (Compute):** Dimensionamento de 2 a 3 VMs (ex: instâncias de 2 vCPUs e 8GB RAM cada) para garantir redundância local de processamento. Custo IaaS estimado: média de R$ 750,00 a R$ 1.500,00/mês.
- **RDBMS Legado (SQL Server):** Licenciamento herdado do cenário *AS-IS*, assumindo o reaproveitamento do contrato corporativo (*Core-Based Licensing* / Licenciamento por Núcleo). A título de estimativa *FinOps*, caso fosse necessário provisionar novas assinaturas (*Subscriptions*) para o mínimo exigido por servidor pela Microsoft (4 *vCores*), o custo de licenciamento seria de aproximadamente R$ 1.200,00 a R$ 1.500,00/mês para a edição *Standard*, e entre R$ 4.500,00 a R$ 5.500,00/mês para a edição *Enterprise*. Contudo, como a organização já absorve este custo no seu *Data Center* atual, o impacto financeiro de aquisição e migração é considerado zero para o orçamento dedicado a esta fase de transição (*MVP)*.
- **Barramento de Mensagens (Kafka ou RabbitMQ):** Hospedagem IaaS dedicada, exigindo de 2 a 3 VMs com armazenamento otimizado para *I/O* (leitura e gravação em disco) para garantir a tolerância a falhas sem estrangular a rede. Estimativa IaaS: média R$ 1.250,00/mês.
- **Bancos de Dados de Leitura (MongoDB e Redis):** Implantação IaaS em instâncias com otimização de memória RAM para suportar leituras em tempo constante O(1). MongoDB (*Community Edition*): média R$ 500,00/mês. Redis: média R$ 250,00/mês.
- **Total Estimado da Infraestrutura Tática:** média de R$ 2.750,00 a R$ 3.500,00/mês (excluindo licenças SQL legadas). Esta fundação garante o desacoplamento imediato com um custo fixo e previsível, mas exige como contrapartida a manutenção manual de sistemas operacionais e *patching* (atualizações de segurança) por parte da equipe.

2.1.4 Governança de Dados, Criptografia e Conformidade com a LGPD (Visão Tática)

Para atender aos requisitos regulatórios da LGPD (*Lei Geral de Proteção de Dados*) e proteger o ecossistema de transações financeiras contra acessos indevidos na fase de transição, a Visão Tática estabelece as seguintes diretrizes de segurança de dados:

- **Criptografia em Trânsito e em Repouso**: Todo o tráfego de rede externo e interno entre o *API Gateway*, as *APIs* e os *Workers* obriga à utilização do protocolo *HTTPS* com *TLS 1.3*. Em repouso, a base de dados relacional (*SQL Server*) utiliza encriptação nativa (*TDE - Transparent Data Encryption*), enquanto os armazenamentos secundários (*Redis* e *MongoDB*) aplicam encriptação em nível de bloco gerida localmente.

- **Isolamento de Rede e Segmentação (Ambiente IaaS/On-premise):** O isolamento físico e lógico do ambiente de transição é assegurado através da segmentação em Zonas de Segurança (*VLANs* ou Sub-redes isoladas). O *API Gateway* e o *WAF* são implantados numa *DMZ* (Zona Desmilitarizada) restrita, sendo os únicos componentes com exposição de IP público. Todos os microsserviços (APIs e *Workers*), o *broker* de mensageria (*Apache Kafka*) e as bases de dados (*SQL Server, MongoDB e Redis*) são implantados em sub-redes estritamente privadas, sem acesso à internet, garantindo que as bases de dados apenas aceitem conexões originadas pelos IPs internos dos contentores autorizados, mitigando o risco de movimentação lateral.

- **Tratamento de Dados Pessoais e Financeiros (LGPD)**: Os dados de clientes e lojistas associados aos lançamentos financeiros passam a adotar o princípio de minimização de dados e mascaramento de campos sensíveis nas camadas de visualização e registro de logs, prevenindo a exposição acidental de *PII* (*Personally Identifiable Information* / Informações de Identificação Pessoal).

- **Políticas de Retenção e Expurgo de Dados (*Data Purging*):**

    - De modo a evitar o crescimento desregulado do armazenamento local e cumprir prazos legais de auditoria financeira, o *MongoDB* (armazenamento a frio) e o *SQL Server* passam a contar com rotinas agendadas (*CronJobs* / scripts de base de dados) parametrizadas com janelas de retenção estritas.

    - Os dados transacionais operacionais ativos mantêm-se por um período padrão regulatório, após o qual são anonimizados ou expurgados de forma irreversível de acordo com as políticas de conformidade da organização.

2.1.5 Atributos de Qualidade: Desempenho, Resiliência e Integridade Transacional (Visão Tática)

Na arquitetura de transição, a plataforma mitiga as restrições físicas do ambiente local (*IaaS*) através de padrões avançados de desacoplamento, garantindo os atributos de qualidade exigidos:

- **Alta Disponibilidade e Resiliência (Comportamento em Falhas e Isolamento)**: A arquitetura baseia-se no princípio de que o domínio de escrita (Frente de Caixa) não deve conhecer o domínio de leitura (Relatórios). Se o Consolidado Diário (*Worker*, *MongoDB* ou *Redis*) ficar indisponível, a *Entry API* continua a registar os Lançamentos do lojista de forma ininterrupta, garantindo 100% de disponibilidade na ponta:

    - ***Cenário de Falha de Comunicação**:* O fluxo utiliza o padrão *Transactional Outbox* aliado ao *CDC (Change Data Capture)*. A *Entry API* processa a venda e grava apenas no *SQL Server* em transação *ACID*. A partir daí, o *CDC* tenta

enviar o evento para o *Kafka*. Se houver falha na comunicação de rede entre a base de dados e o *Kafka*, a transação principal já foi salva com sucesso no *SQL Server* (evitando a perda de dados). Quando a comunicação é restabelecida, o *CDC* retoma a leitura a partir do seu último ponto seguro (*offset*), garantindo a entrega ao sistema de leitura sem duplicidade.

- **Escalabilidade e Comprovação de Desempenho (Carga e 50 RPS)**: A estratégia para garantir o processamento de 50 Requisições Por Segundo sem perdas apoia-se no algoritmo *Token Bucket* (balde de tokens) configurado no *API Gateway*, que atua como barreira arquitetural:
    - ***Evidência e Testes de Desempenho**:* A validação deste limiar será atestada através de Testes de Carga (utilizando ferramentas como *JMeter*). A esteira de testes injetará intencionalmente 100 RPS contra o sistema. O teste evidenciará que o *API Gateway* barra o tráfego excedente (devolvendo código HTTP 429 - *Too Many Requests*) e entrega exatamente o teto de 50 RPS à *Entry API*. Como as requisições válidas são processadas de forma isolada pela base relacional, sem bloqueios (*locks*) provenientes de consultas de saldo, a arquitetura assegura matematicamente que o volume de perda de transações aceites é nulo (0%), cumprindo com extrema folga a tolerância máxima exigida de 5%.
- **Tratamento de falhas, duplicidades e inconsistências**: Em sistemas distribuídos com consistência eventual, a garantia da integridade financeira exige mecanismos de salvaguarda específicos para o tratamento do ciclo de vida dos eventos:
    - **Eventos Não Processados (*Dead-Letter Topics*):** O *Apache Kafka* garante a entrega, mas falhas lógicas no *Consolidation Worker* (ex: erro de *parsing* ou *timeout* ao gravar no *MongoDB*) podem impedir o processamento de uma mensagem. Para evitar o bloqueio da partição, implementa-se o padrão *Retry/Dead-Letter Topic* (DLT). Se o evento falhar repetidamente após as tentativas configuradas (*retries* locais), ele é desviado para um tópico de DLT no *Kafka*. Um alerta é gerado no *Grafana* para que a equipe de operações analise a anomalia sem interromper o fluxo de novas vendas.
    - **Mensagens Duplicadas (Idempotência)**: Como o *Kafka* opera com a premissa de entrega *at-least-once* (pelo menos uma vez), o *Worker* está desenhado para ser estritamente idempotente. Cada lançamento financeiro gerado pela *Entry API* possui um identificador único universal (*Transaction ID*). Antes de atualizar o saldo no *Redis*, o *Worker* consulta uma coleção de controlo no *MongoDB* para verificar se aquele *Transaction ID* já foi processado. Se a mensagem for reentregue por uma falha transitória de rede, o *Worker* descarta o evento silenciosamente, impedindo a duplicação de valores no saldo do cliente.
    - **Resolução de Inconsistências nos Saldos Consolidados**: Devido à natureza assíncrona, falhas no sincronismo podem gerar divergências temporárias entre a base de escrita e a de leitura. Para mitigar este risco, é implementado um *Job* de Reconciliação Noturna (*CronJob*). Este processo em lote (*batch*) calcula o saldo absoluto a partir da base de dados relacional (*SQL Server* - fonte da verdade) e compara-o com a base de leitura (*MongoDB/Redis*). Caso seja detectada uma quebra de integridade,

o *Job* invalida o *cache* e reconstrói o saldo automaticamente (padrão *Snapshot Rebuild*), garantindo que o sistema inicie o dia seguinte com consistência absoluta.

2.1.6 Estratégia de Monitorização e Observabilidade (Visão Tática) Para abandonar a prática frágil de análise de ficheiros de texto locais (.log) do cenário legado, a arquitetura de transição implementa uma estratégia de Observabilidade Proativa baseada nos três pilares fundamentais: Métricas, *Logs* Estruturados e *Traces* Distribuídos.

- **Instrumentação Unificada:** Toda a aplicação (APIs e *Workers*) é instrumentada utilizando o padrão *OpenTelemetry*, garantindo que o código não fica acoplado a uma ferramenta específica de monitorização.
- **Pilares de Monitorização (Stack *Open-Source*):**
    - **Métricas e Dashboards:** O *Prometheus* recolhe métricas de infraestrutura (*CPU/RAM*) e de negócio, enquanto o *Grafana* centraliza a visualização. Os painéis do *Grafana* monitorizarão proativamente dois indicadores críticos: a taxa de rejeição no *API Gateway* (para validar a eficácia do *Rate Limit* de 50 RPS) e o *Consumer Lag* do *Apache Kafka* (para garantir que o *Worker* está a processar as vendas à mesma velocidade que entram).
    - **Logs Centralizados:** Os registros de aplicação passam a ser formatados em *JSON* estruturado e centralizados numa *stack* ELK (*Elasticsearch, Logstash, Kibana*) ou equivalente local, permitindo pesquisas rápidas e correlacionadas sem acesso *RDP/SSH* aos servidores.
    - **Traces Distribuídos:** Cada transação iniciada no *API Gateway* recebe um *Correlation ID* único, permitindo rastrear o ciclo de vida completo do lançamento financeiro através do domínio de escrita, do barramento de eventos e do domínio de leitura.

SEÇÃO 2.2: VISÃO TARGET - ARQUITETURA CLOUD-NATIVE (AZURE)

A **Visão Target** representa o estado de maturidade arquitetural definitiva. O sistema é integralmente migrado para o ecossistema *Microsoft Azure*, utilizando serviços geridos no modelo *PaaS* (*Platform as a Service* / Plataforma como Serviço) e computação *Serverless* (Computação sem Servidor). Esta abordagem elimina o esforço de sustentação de infraestrutura física, eleva a postura de cibersegurança e habilita a integração contínua de novas capacidades de negócio.

A escolha estratégica pelo ecossistema *Microsoft Azure* visa exemplificar a adoção de tecnologias geridas de alta performance. Contudo, a padronização do empacotamento do código em contentores universais assegura o total agnosticismo da solução. Esta decisão garante que a plataforma mantenha a sua independência em relação ao provedor de infraestrutura, protegendo a organização contra a dependência tecnológica exclusiva e conferindo a flexibilidade necessária para portar a operação para outras nuvens públicas no futuro, caso o negócio assim o exija.

A figura 9 apresenta a arquitetura foi dividida em domínios isolados, permitindo escalabilidade independente.

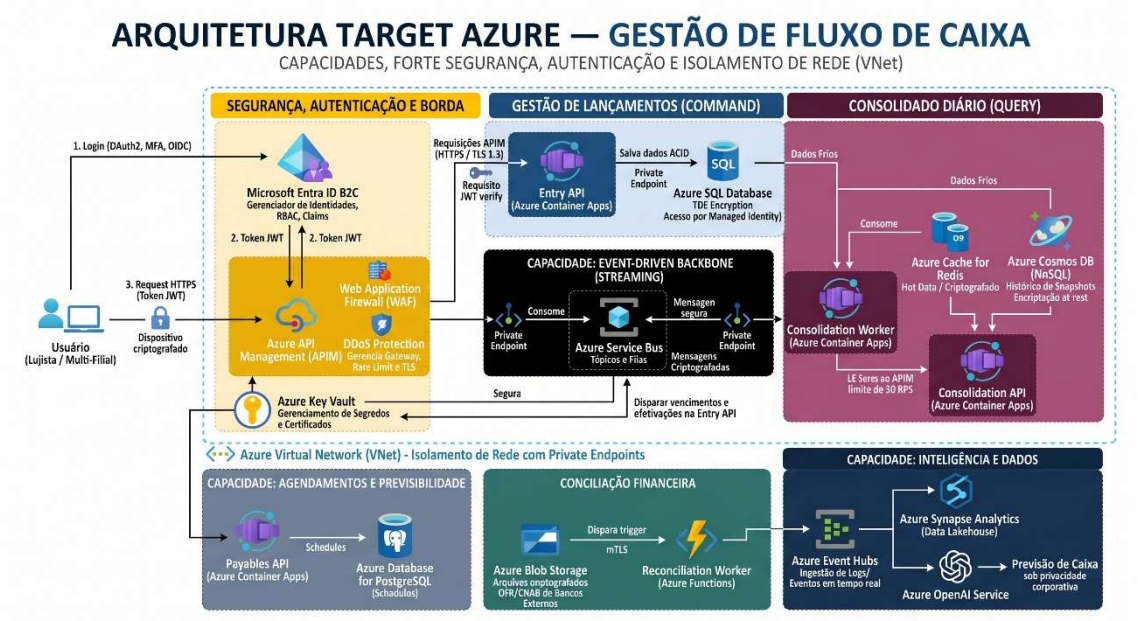


Figura 9– Arquitetura Visão Target Azure – Gestão de Fluxo de Caixa

A infraestrutura foi dividida em áreas de capacidades independentes, protegidas por rigoroso isolamento de rede:

1. **Segurança, Autenticação e Borda:** Utilização do *Microsoft Entra ID B2C* para a gestão de identidades e implementação de *RBAC* (*Role-Based Access Control* / Controlo de Acesso Baseado em Papéis). O isolamento de rede é assegurado através de uma *Azure VNet* (*Virtual Network* / Rede Virtual) com *Private Endpoints* (Pontos de Extremidade Privados), impedindo a exposição das APIs à internet pública. A fronteira de segurança é gerida pelo *Azure API Management (APIM)* acoplado a um *WAF* para mitigação de ataques *DDoS* (Negação de Serviço Distribuída). As credenciais de sistema são dinamicamente injetadas via *Azure Key Vault* (Cofre de Segredos).

2. **Gestão de Lançamentos (*Command*):** Implementação da *Entry API* sobre *Azure Container Apps*, permitindo *Auto-scaling* (Escalabilidade Automática) baseada em métricas de carga (*KEDA*). A persistência relacional é delegada ao *Azure SQL Database*, que fornece suporte nativo a criptografia de dados em repouso (*TDE - Transparent Data Encryption*).

3. **Barramento Orientado a Eventos (*Streaming*):** Substituição do *Kafka* gerido manualmente (IaaS) pelo *Azure Service Bus*, um serviço *PaaS* totalmente gerido, assegurando a entrega transacional e ordenada de mensagens (*Topics e Filas*) com alta disponibilidade nativa.

4. **Consolidado Diário (*Query*):** A agregação dos eventos assíncronos é realizada por *Workers* baseados em contentores que atualizam, simultaneamente, o *Azure Cosmos DB* (serviço *NoSQL* de distribuição global e latência na ordem dos milissegundos) e o *Azure Cache for Redis*.

5. **Capacidades Avançadas de Retaguarda (Inovação):** Introdução da automação de conciliação financeira através do *Azure Functions* (*Serverless*) acionada por *triggers* (gatilhos) originados no *Azure Blob Storage* quando ficheiros bancários são

recebidos. Ingestão paralela de telemetria no *Azure Event Hubs* para alimentar o *Azure Synapse Analytics* (*Data Lakehouse*), viabilizando previsibilidade de caixa através do *Azure OpenAI Service* (Inteligência Artificial).

O Mapeamento de Valor e Capacidades Técnicas (Visão Target) visa clarificar os ganhos operacionais proporcionados por esta nova arquitetura, apresentando um quadro que correlaciona cada domínio de negócio à respectiva capacidade técnica implementada na nuvem e evidenciando o valor direto entregue à operação diária do comerciante e à organização:

Domínio de Negócio e Capacidade	Componentes Técnicos (Azure)	Valor Entregue à Operação (ROI Técnico)
Segurança e Borda (Zero-Trust)	Entra ID B2C, WAF, API Management (APIM), Virtual Network (VNet)	**Proteção Corporativa:** Mitiga vulnerabilidades legadas, garantindo que o sistema suporte o limite rigoroso de 50 RPS sem sucumbir a ataques maliciosos (DDoS/L7) e sem expor as APIs à internet.
Operações de Frente de Caixa	Entry API (Container Apps) + Azure SQL Database	**Lançamentos sem falhas:** O *Auto-scaling* absorve picos de venda de forma elástica. O comerciante realiza registros rápidos e com rigorosa encriptação (TDE), estancando os travamentos sistémicos.
Transporte Assíncrono (Eventos)	Azure Service Bus (Barramento de Serviços)	**Tolerância a Falhas:** Substitui filas locais (como o Kafka gerido manualmente) por um serviço PaaS, garantindo a retenção da transação caso os serviços de leitura falhem.
Relatórios Gerenciais (Consolidado)	Worker/API (Container Apps) + CosmosDB + Azure Cache for Redis	**Visibilidade em Tempo Real:** O comerciante consulta saldos de forma instantânea através do *Redis*, eliminando a concorrência e a lentidão na base de dados durante o horário comercial.
Automação Financeira (Conciliação)	Azure Blob Storage + Azure Functions (Serverless)	**Eficiência Administrativa:** Elimina o esforço manual cruzando automaticamente os lançamentos de caixa com os ficheiros bancários (OFX/CNAB) recebidos.
Inovação e Inteligência de Dados	Azure Synapse (Data Lake) + Azure OpenAI Service	**Previsibilidade:** Processa o fluxo histórico do comerciante para fornecer dicas financeiras baseadas em IA (alertas de quebra de caixa), habilitando novos modelos de receita para a plataforma.

2.2.1 Representação Arquitetural (C4 Model) - Visão Target

Para consolidar o desenho da infraestrutura definitiva e a sua adoção do padrão *Zero-Trust*, a arquitetura *Cloud-Native* foi mapeada sob o *C4 Model*. As visões a seguir evidenciam o isolamento de rede através de *Private Endpoints* e a exploração total do ecossistema de serviços geridos (*PaaS*) da *Azure*.

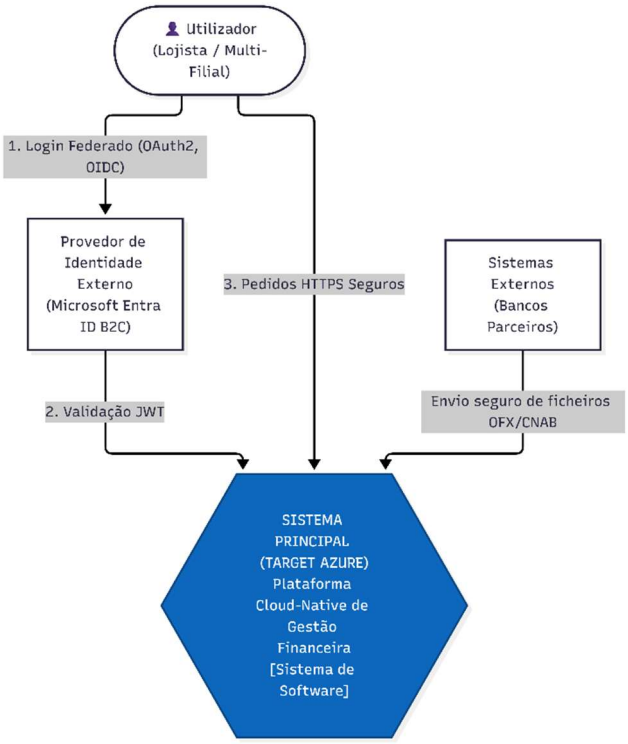


Figura 10– Diagrama de Contexto

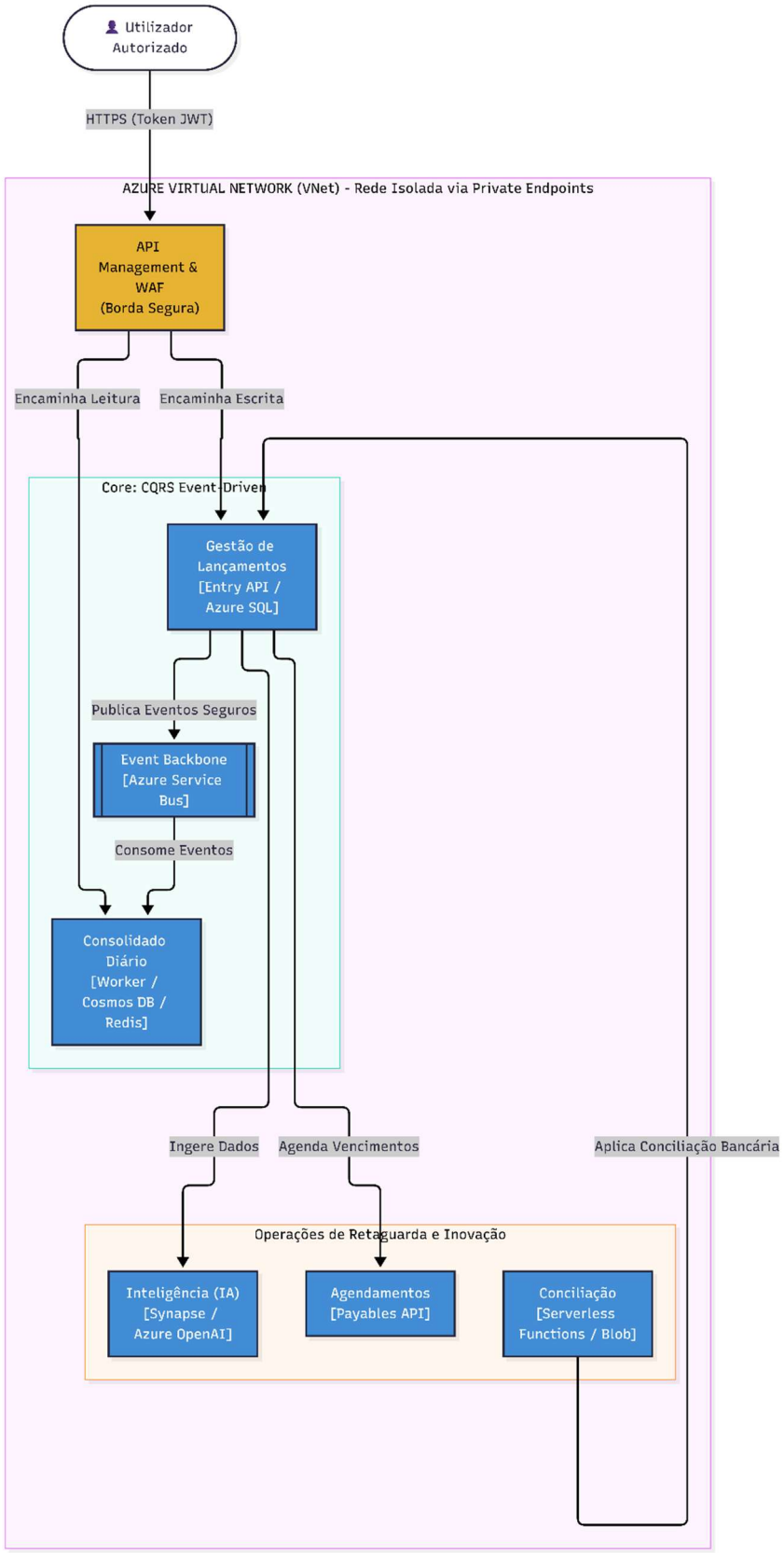


Figura 11– Diagrama de Container

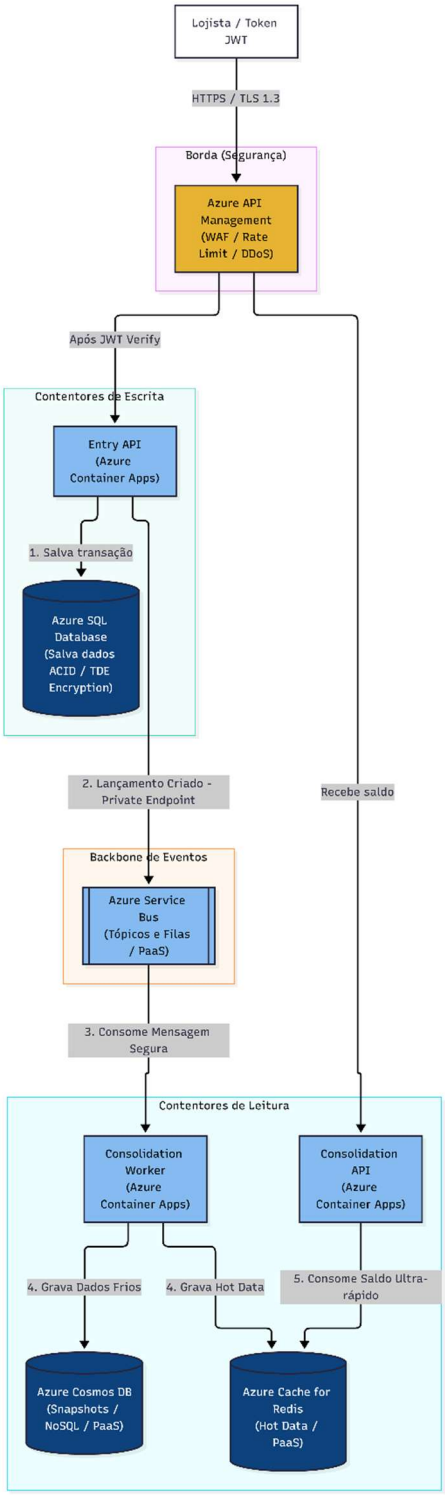


Figura 12– Diagrama de Componentes – Parte 1: Fluxo de Entrada

2.2.2 Requisitos (Visão Target)

A Visão Target expande os requisitos táticos para incorporar segurança corporativa, automação e adoção integral de serviços geridos na nuvem:



Requisitos funcionais:

- **RF-01 (Registro de Movimentação):** O sistema deve fornecer uma API (Entry API) capaz de receber, validar e registar lançamentos financeiros (débitos e créditos) iniciados pelo lojista.
- **RF-02 (Ingestão Legada - CDC):** O sistema deve capturar de forma assíncrona, através de *Change Data Capture* (CDC), as alterações realizadas na base de dados legada e transformá-las em eventos de negócio.
- **RF-03 (Consolidação de Saldo):** O sistema deve possuir um processo em *background* (*Worker*) que consuma os eventos de lançamentos e processe matematicamente a consolidação do saldo financeiro diário por estabelecimento.
- **RF-04 (Consulta de Relatório):** O sistema deve fornecer uma API (Consolidation API) para que o lojista consulte o saldo consolidado atualizado, de forma independente do fluxo de escrita.
- **RF-05 (Autenticação Federada):** O sistema deve autenticar os utilizadores (lojistas e multifiliais) utilizando um provedor de identidade centralizado (Microsoft Entra ID B2C) gerando *tokens* JWT com base em *Claims* e RBAC (Role-Based Access Control).
- **RF-06 (Agendamento de Pagáveis):** O sistema deve permitir o agendamento de contas a pagar através de uma API dedicada (Payables API) e base de dados relacional própria.
- **RF-07 (Conciliação Bancária Automatizada):** O sistema deve processar arquivos padrão bancário recebidos em *storage*, disparando automaticamente uma rotina de conciliação para cruzar o caixa com a conta bancária.

Requisitos não-funcionais:

- **RNF-01 (Desempenho e Throughput):** As APIs devem suportar picos de tráfego de até 50 Requisições Por Segundo (RPS) sem degradação de tempo de resposta. As requisições de consulta de saldo devem ter complexidade temporal O(1) (Tempo constante).
- **RNF-02 (Resiliência e Tolerância a Falhas):** A perda máxima tolerável de eventos/transações é fixada no rigoroso limite de 5%. O sistema de mensageria deve atuar como *buffer* (retenção em disco/fila) para garantir que picos de carga não resultem em perda de dados caso as APIs de leitura estejam indisponíveis.
- **RNF-03 (Isolamento de Domínio / CQRS):** A disponibilidade do domínio de escrita (Gestão de Lançamentos) não pode ser impactada pela indisponibilidade ou lentidão do domínio de leitura (Consolidado Diário), eliminando o "Acoplamento Síncrono Total" do legado.
- **RNF-04 (Segurança Zero-Trust):** A arquitetura deve implementar isolamento total de rede pública. Todo o tráfego interno entre microsserviços e bases de dados deve ocorrer em uma Virtual Network (VNet) através de *Private Endpoints*.
- **RNF-05 (Proteção e Criptografia):** A borda deve ser protegida por Web Application Firewall (WAF) mitigando ataques L7. Os dados em trânsito devem

usar mTLS/TLS 1.3 e dados em repouso exigem criptografia TDE (*Transparent Data Encryption)*. Acesso a bancos de dados deve usar identidades gerenciadas (sem *passwords* em código).

- **RNF-06 (Elasticidade):** O processamento de retaguarda (APIs e Workers) deve escalar horizontalmente de forma automática (Auto-scaling / *Scale-to-Zero*) com base no volume de eventos da fila de mensageria ou carga de CPU.

Requisitos técnicos:

- **RT-01 (Stack de Desenvolvimento):** O código fonte das APIs e do Worker deve ser escrito utilizando a linguagem de domínio do time de engenharia(programação).
- **RT-02 (Bancos de Dados Poliglotas):** O armazenamento deve respeitar a especialidade de cada domínio:
    - *Command:* SQL Server (garantia transacional ACID).
    - *Query (Hot Data):* Redis Cache (para consulta ultrarrápida do saldo).
    - *Query (Cold Data):* MongoDB (NoSQL) para armazenamento de histórico de *snapshots*.
- **RT-03 (Mensageria e Eventos):** Substituição do *Kafka* por *Azure Service Bus* e adoção do *Azure Event Hubs* para ingestão de telemetria em tempo real.
- **RT-04 (Conteinerização):** A execução e orquestração local (ambiente de desenvolvimento) de toda a infraestrutura e código deverá ser feita exclusivamente via Docker/Docker Compose.
- **RT-05 (Cloud Provider PaaS):** Toda a infraestrutura Target será migrada para serviços geridos (PaaS) e Serverless no ecossistema Microsoft Azure ou para o provedor em nuvem de decisão da empresa , com a restrição de manter o código de aplicação agnóstico (desacoplado do *vendor*) via uso de *Containers*.
- **RT-06 (Mapeamento de Serviços PaaS):**
    - *API Gateway & Rate Limiting:* Azure API Management (APIM).
    - *Computação (APIs e Workers):* Azure Container Apps (para o CQRS central) e Azure Functions (para a Conciliação via *Trigger)*.
    - *Bancos de Dados Gerenciados:* Azure SQL Database, Azure Cosmos DB e Azure Cache for Redis.
    - *Mensageria PaaS:* Azure Service Bus (substituindo o Kafka) e Azure Event Hubs para ingestão maciça de logs.
- **RT-07 (Observabilidade Unificada):** É estritamente proibido o armazenamento de logs textuais em disco local. Toda a aplicação deve ser instrumentada com **OpenTelemetry**, enviando Traces, Logs e Metrics para o Grafana (na Visão Tática) e migrando para o Azure Log Analytics/Application Insights na Visão Target.

- **RT-08 (Gestão de Segredos):** As credenciais, chaves de API e certificados da arquitetura Target deverão ser obrigatoriamente injetados no tempo de execução através do Azure Key Vault.

2.2.3 Estimativa de Custos, Metodologia de Cálculo e Elasticidade (Visão Target)

Na proposta de mensurar os custos mensais da Visão Target, a estimativa mensal está baseada em *consumo Pay-As-You-Go:* Na Visão Target, a arquitetura transita para um modelo de custos 100% elástico (OpEx / *Pay-As-You-Go*). Os cálculos foram balizados pelos parâmetros da calculadora oficial da Microsoft (*Azure Pricing Calculator* - Região *East US*), utilizando como premissa matemática o teto exigido de 50 Requisições Por Segundo (RPS):

- **Segurança e Borda (Azure API Management & WAF):** Utilização do *Tier Standard* (necessário para isolamento *VNet* e *Rate Limiting*). A metodologia foca-se num faturamento fixo mensal baseado em 1 unidade de escala operando 730 horas. Estimativa: média de R$ 3.500,00/mês.
- **Computação Elástica (Azure Container Apps):** Utilização do plano *Consumption*. **Metodologia:** O pico de 50 RPS ao longo do mês gera um tráfego de aproximadamente 130 milhões de requisições. As réplicas foram dimensionadas com 1 vCPU e 2GB RAM. Como o faturamento é exclusivo por milissegundo de uso ativo, a adoção do *Scale-to-Zero* (redução a zero instâncias durante a madrugada) reduz drasticamente o desperdício. Estimativa: média entre R$ 750,00 a R$ 1.250,00/mês.
- **Bancos de Dados Gerenciados (Azure SQL & Cosmos DB):** O *Azure SQL* opera no *Tier Serverless Gen5*, faturado estritamente por segundo de computação ativa (com *Auto-pause* habilitado para momentos de inatividade). O *Cosmos DB* opera no modelo *Serverless*, faturado pela métrica de *Request Units* (RU/s) efetivamente consumidas nas operações de gravação e extração. Estimativa conjunta: média entre R$ 750,00 a R$ 3.500,00/mês.
- **Mensageria e Cache (Azure Service Bus & Redis):** O *Service Bus* no *Tier Standard* tem o seu custo derivado de uma taxa base somada a um valor centesimal por cada milhão de operações. O *Azure Cache for Redis* opera no *Tier Basic C1* (1GB), faturado por hora provisionada. Estimativa conjunta: média de R$ 250,00/mês.
- **Operações de Retaguarda e IA (Functions, Synapse, Blob, OpenAI):** Custos estritamente orientados a eventos, oscilando desde isenções (faixas gratuitas de execução Serverless) a uma média de R$ 1.000,00 a R$ 2.000,00/mês (fortemente atrelada à volumetria de processamento de *tokens* pelo LLM nas previsões de caixa).
- **Total Estimado Target:** média entre R$ 6.500,00 a R$ 10.500,00/mês.

2.2.4 Governança, Criptografia Avançada e Expurgo Automatizado (Visão Target)

Na arquitetura definitiva *Cloud-Native* na *Azure*, a proteção de dados financeiros e de clientes atinge o nível máximo de conformidade regulatória e autonomia operacional através de serviços geridos:

- **Criptografia de Ponta a Ponta com Gestão de Chaves Corporativas**:
    - O tráfego de rede utiliza rigorosamente *mTLS* (*Mutual TLS*) e *TLS 1.3*.
    - Todas as chaves de encriptação em repouso para o *Azure SQL Database*, *Azure Cosmos DB* e *Azure Blob Storage* deixam de ser geridas por chaves padrão da nuvem e passam a adotar o modelo de chaves geridas pelo cliente (*CMK - Customer-Managed Keys*), integradas de forma centralizada e auditável com o *Azure Key Vault*.
- **Conformidade Rigorosa com a LGPD e Mascaramento Dinâmico**:
    - Implementação do *Azure SQL Data Masking* para ocultar automaticamente informações sensíveis de cartões, contas e dados pessoais de clientes nas consultas executadas por utilizadores sem privilégios administrativos.
    - Aplicação de políticas de classificação de dados sensíveis para monitorização contínua de acessos indevidos através do *Microsoft Defender for Cloud*.
- **Gestão de Ciclo de Vida e Expurgo Automatizado de Dados (*Data Lifecycle Management - DLM*)**:
    - **No *Azure Cosmos DB* (Dados Frios / Histórico)**: Utilização nativa da propriedade *TTL* (*Time-To-Live*), configurada diretamente nos contentores para que documentos transacionais e históricos expirados face ao prazo legal de retenção sejam automaticamente eliminados pelo motor do banco de dados, sem intervenção manual ou scripts externos.
    - **No *Azure Blob Storage* (Arquivos Fiscais / CNAB / OFX):** Aplicação de regras automáticas de expurgo baseadas no ciclo de vida do objeto, garantindo a exclusão programada ou a transição para armazenamento de arquivo morto imutável (*WORM - Write Once, Read Many*) para auditorias fiscais obrigatórias, mitigando riscos de retenção infinita e desperdício financeiro (*FinOps*).

2.2.5 Atributos de Qualidade: Desempenho, Resiliência e Integridade Transacional (Visão Target)

Na arquitetura definitiva em nuvem, a plataforma herda as capacidades nativas de infraestrutura distribuída do ecossistema *Microsoft Azure*, automatizando a resiliência a desastres e a elasticidade de tráfego.

- **Alta Disponibilidade e Resiliência (Tolerância a Desastres e Isolamento):** A comunicação entre serviços consolida-se de forma assíncrona gerida através do *Azure Service Bus*. Se o ecossistema de leitura (*Cosmos DB* ou *Consolidation Worker*) sofrer uma interrupção crítica, a *Entry API* prossegue com as gravações, acumulando os lançamentos de forma durável nas filas encriptadas do *Service Bus*:
    - ***Cenário de Falha de Comunicação**:* Falhas transitórias de rede (ex: micro-indisponibilidades ao tentar publicar no *Service Bus*) são mitigadas na origem por políticas de resiliência a nível de código — implementando

padrões de *Retry* com *Exponential Backoff* e *Circuit Breaker* (via bibliotecas como *Polly*). Adicionalmente, caso um evento cause erros sucessivos de processamento no *Worker*, a mensagem não bloqueia o sistema; é desviada para uma *Dead-Letter Queue* (*DLQ*) para posterior tratamento automatizado, garantindo a disponibilidade dos serviços independentemente da saúde dos seus vizinhos de rede. A infraestrutura passa também a contar com *Availability Zones* (*AZs*), assegurando operação mesmo em caso de falha física de um *Data Center* da Microsoft.

- **Escalabilidade e Comprovação de Desempenho (Elasticidade Nativa):** A restrição de 50 RPS é aplicada a nível perimetral pelo *Azure API Management* através de políticas rigorosas de <rate-limit>:
    - ***Evidência e Testes de Desempenho**:* Diferentemente do cenário tático de recursos estáticos, o ambiente Target comprova a sua capacidade de carga através de *Auto-scaling* orientado a eventos (*KEDA*) nativo no *Azure Container Apps*. A política de testes de carga contínua, integrada nas esteiras de *CI/CD* (*Azure Load Testing*), garante que a *Entry API* escale novas réplicas horizontalmente de forma automática antes que a métrica de *CPU* alcance 70%. Simultaneamente, o *Azure SQL* operará no modelo *Serverless* com provisionamento elástico de *vCores*. Esta dinâmica arquitetural atesta (e gera telemetria em tempo real no *Azure Log Analytics*) que todas as 50 transações por segundo autorizadas pela borda serão processadas com latência sustentada na casa dos milissegundos (Percentil 99 < 150ms para escrita e < 10ms para leitura no *Redis*), blindando o sistema contra estrangulamento e respeitando a tolerância de perda exigida no desafio de forma incontestável.
- **Tratamentos de falhas, duplicidades e inconsistências:** Na arquitetura madura em nuvem, a gestão de anomalias tira partido das capacidades geridas da Azure para automatizar a correção de estado e garantir a integridade financeira em tempo real:
    - **Eventos Não Processados (*Native Dead-Letter Queues - DLQ*)**: Ao contrário do *Kafka*, o *Azure Service Bus* possui gestão nativa de mensagens não processáveis. Se um evento exceder a contagem máxima de entregas configurada (*Max Delivery Count*) devido a uma falha no *Consolidation Worker*, a infraestrutura move a mensagem automaticamente para a fila *DLQ*. Uma *Azure Function* acionada por eventos monitoriza a *DLQ* e aplica políticas de autocorreção ou dispara alertas críticos via *Azure Monitor*, garantindo rastreabilidade total de transações pendentes.
    - **Mensagens Duplicadas (Deteção Nativa)**: A mitigação de duplicidades atua em duas camadas. Primeiramente, o *Azure Service Bus* é configurado com a funcionalidade nativa de *Duplicate Detection*, que rejeita automaticamente mensagens com o mesmo *MessageId* (*Transaction ID*) dentro de uma janela de tempo definida. Na segunda camada, o *Consolidation Worker* interage com o *Azure Cosmos DB* utilizando concorrência otimista (*Optimistic Concurrency Control* via *ETags*) e operações de *Upsert* condicionais, garantindo matematicamente que uma

transação retardada não sobrescreva um saldo mais recente, mantendo a operação *thread-safe*.

- **Resolução de Inconsistências nos Saldos Consolidados**: Em vez de aguardar por processos noturnos em lote, a arquitetura Target introduz a reconciliação orientada a eventos. Um *Reconciliation Worker* (implementado em *Azure Functions*) atua como auditor contínuo. De forma amostral e periódica ao longo do dia, ele cruza os dados do *Azure SQL Database* com o estado materializado no *Azure Cosmos DB*. Se uma anomalia for detectada (por exemplo, devido a um atraso massivo na replicação), a função dispara um comando de compensação ou aciona o reprocessamento da janela de tempo afetada (*Event Replay*), restaurando o saldo do *Azure Cache for Redis* de forma transparente para o lojista.

2.2.6 Observabilidade Proativa e Governança de Custos (FinOps) (Visão Target) Na arquitetura definitiva em nuvem, a monitorização funde-se com a gestão financeira. O modelo de custo elástico exige que a observabilidade atue não apenas na deteção de falhas técnicas, mas também na prevenção de anomalias de faturação.

- **Monitorização de Aplicação e Telemetria Nativa:** A stack *open-source* transitória é substituída pelo Azure Monitor e Application Insights. Através da funcionalidade de *Application Map*, a equipe de engenharia obtém um diagrama de topologia em tempo real, visualizando latências e gargalos entre o *Azure API Management*, *Container Apps*, *Service Bus* e as bases de dados geridas.
- **Alertas Inteligentes e Mapeamento de Risco:** São configurados alertas acionáveis baseados em limiares (*Thresholds*) dinâmicos. A equipe de resposta a incidentes é notificada automaticamente via *Webhooks/Microsoft Teams* caso ocorram anomalias no padrão transacional (ex: pico inesperado de mensagens desviadas para a *Dead-Letter Queue - DLQ* do *Service Bus* ou latência de gravação no *Cosmos DB* superior a 15ms).
- **Monitorização Contínua de Custos (Estratégia FinOps):** A observabilidade financeira é garantida através do Azure Cost Management. São estabelecidos orçamentos mensais (*Budgets*) com alertas preventivos disparados aos 70%, 90% e 100% do consumo estimado. Adicionalmente, a monitorização de utilização do *Auto-scaling* (*KEDA*) garante que os contentores efetivamente escalem para zero (*Scale-to-Zero*) fora do horário comercial, assegurando o Retorno sobre o Investimento (*ROI*) técnico e financeiro planeado.

SEÇÃO 2.3: ESTRATÉGIA DE TRANSIÇÃO E MIGRAÇÃO (ZERO-DOWNTIME)

Para garantir que a passagem da Visão Tática (*IaaS/MVP*) para a Visão Target (*Cloud-Native Azure*) ocorra sem qualquer impacto para o lojista no momento da venda, a estratégia de *Cutover* (virada de chave) foi desenhada com a premissa estrita de Zero-Downtime.

A substituição das APIs de ingestão e a movimentação do histórico de dados não ocorrerão de forma abrupta. Em vez disso, será adotado o padrão de coexistência temporal, onde ambos os ambientes processam os dados simultaneamente até que a consistência do ambiente Target seja validada.

A tabela a seguir detalha o mapeamento cronológico da transição:

Tabela de Cronograma de Migração Tático ➔ Target			
Fase Cronológica	**Ação Arquitetural e Substituição de APIs**	**Migração de Dados (MongoDB ➔ Cosmos DB)**	**Estratégia para Zero-Downtime na Virada de Chave**
Fase 1: Provisionamento e Sincronização Inicial	Implantação da infraestrutura Target (Azure APIM, Service Bus, Container Apps) sem receber tráfego real. A API Tática continua a ser a única a receber chamadas do lojista.	Utilização do *Azure Database Migration Service* (ou rotina de extração via *Worker*) para migrar toda a base de dados fria existente do *MongoDB Tático* para o *Azure Cosmos DB*.	**Background Sync:** A cópia dos dados históricos ocorre em segundo plano, sem aplicar *locks* ou concorrer com as operações ativas no ambiente Tático.
Fase 2: Duplo Barramento (Dual-Write)	O *CDC* (Change Data Capture) passa a duplicar a publicação de eventos: publica tanto no *Apache Kafka* (Tático) quanto no *Azure Service Bus* (Target).	O *Consolidation Worker* Target entra em ação, consumindo os novos eventos do *Service Bus* e atualizando o *Cosmos DB* e o *Azure Cache for Redis* em tempo real.	**Coexistência:** Ambos os sistemas calculam o saldo diário em paralelo. A equipe de engenharia valida se o saldo Target coincide exatamente com o saldo Tático, sem que o utilizador perceba.
Fase 3: Substituição Gradual (*Canary Release*)	O *API Gateway* Tático é substituído como fronteira principal pelo *Azure API Management* (Target), que passa a atuar como roteador global de tráfego.	Os dados já estão 100% sincronizados. O *Cosmos DB* atua agora como a base histórica oficial e definitiva.	**Roteamento Dinâmico:** O *Azure APIM* desvia inicialmente 10% do tráfego das requisições para a *Entry API Target* (Azure), mantendo 90% na *Entry API Tática*. Se não houver erros, a percentagem sobe gradualmente para 100%.
Fase 4: Virada de Chave Final (*Cutover*)	A *Entry API Target* e a *Consolidation*	O *MongoDB* e o *Redis* Táticos	**Comutação Transparente:** O

	API Target assumem 100% das operações de escrita e leitura de saldos.	deixam de receber atualizações e têm os acessos de escrita revogados (*Read-only*).	lojista nunca perde a conexão. A mudança de versão da API ocorre apenas a nível de roteamento perimetral, sem alteração dos *endpoints* do cliente.
Fase 5: Descomissionamento (*Sunset*)	Remoção de toda a infraestrutura IaaS Tática (Contentores locais, Entry API legada, Consolidation Worker legado).	O *MongoDB* Tático e o disco do *Kafka* são expurgados após um período de quarentena de 15 dias (política de Rollback seguro).	**Limpeza FinOps:** Redução imediata de custos de infraestrutura redundante após atestado de estabilidade do ambiente Azure.

Para assegurar a confiabilidade deste cronograma e impedir qualquer regressão na operação financeira, a progressão entre as fases é estritamente governada por um Validador de Integridade automatizado. Durante as fases de *Dual-Write* e *Canary Release*, um *Job* de Reconciliação atua como auditor contínuo em segundo plano, cruzando matematicamente os saldos gerados pela arquitetura Tática com os saldos materializados na arquitetura Target a cada janela de 5 minutos. O avanço para a Fase 4 (*Cutover* definitivo) só é autorizado após o validador atestar 100% de paridade de dados (*Data Parity*) durante 72 horas ininterruptas, aliado à comprovação de latência P99 abaixo da meta estipulada no *Azure Monitor*. Caso o validador detecte qualquer divergência de cêntimos ou degradação de performance no tráfego do novo ambiente, o *Azure API Management* executa um *Rollback* automatizado, redirecionando o tráfego instantaneamente de volta para o ambiente Tático sem perda de requisições, garantindo que o risco de negócio seja zero durante toda a janela de modernização.

Seção 3: Justificativa

A decisão de dividir a modernização do sistema em duas etapas (Visão Tática e Visão Target) fundamenta-se na necessidade de estancar as dores urgentes do negócio sem incorrer nos altíssimos riscos e na demora de uma migração total. A adoção da Visão Tática com os padrões CQRS e arquitetura orientada a eventos (Event-Driven) resolve o problema crítico de acoplamento síncrono e travamentos de banco de dados do ambiente legado. Esta abordagem isola a capacidade de Gestão de Lançamentos da de Consolidado Diário, utilizando CDC (Change Data Capture) para a ingestão contínua de dados sem impactar a operação.

Para a arquitetura definitiva, a Visão Target justifica-se pela meta estratégica de atingir a maturidade máxima em resiliência, segurança e escalabilidade, transferindo o fardo operacional da gestão de infraestrutura para serviços gerenciados (PaaS e Serverless). Esta

fundação técnica cumpre com rigor os requisitos não-funcionais estabelecidos: suporte a 50 RPS (Requisições Por Segundo), proteção contra perdas e latência baixíssima para relatórios.

Adicionalmente, justifica-se a substituição do armazenamento legado de *logs* em arquivos de texto (que causavam paradas por disco cheio) pela adoção do OpenTelemetry. Este padrão de observabilidade permitirá a instrumentação unificada da aplicação, gerando *logs, traces* distribuídos e métricas precisas. Na Visão Tática, esses dados serão consolidados e visualizados no Grafana (e posteriormente no Azure Log Analytics/Application Insights na Visão Target), garantindo visibilidade em tempo real sobre a saúde do sistema e reduzindo o tempo de resposta a incidentes.

Ainda no contexto da Visão Tática, embora a arquitetura de referência ilustre o *Apache Kafka*, a adoção do RabbitMQ é documentada como uma alternativa viável e de menor atrito operacional para o MVP. Como a volumetria exigida é de apenas 50 RPS, o Kafka traz uma complexidade de infraestrutura desnecessária para este momento, já que é desenhado para suportar centenas de milhares de requisições. O RabbitMQ resolve perfeitamente este cenário: atuando como um "Smart Broker" (via protocolo AMQP), ele utiliza Filas Duráveis e confirmações de entrega nativas para garantir a meta de perda máxima de 5% com um esforço de configuração significativamente menor.

No entanto, esta escolha impacta a estratégia da Visão Target. Caso o Kafka seja mantido no MVP, ele não exigiria substituição na nuvem definitiva, destacando-se por ser uma tecnologia totalmente agnóstica. Na Azure, por exemplo, o Kafka operaria de forma nativa e sem necessidade de alteração de código através do *Azure Event Hubs* (que oferece um *endpoint* 100% compatível com a API do Kafka) ou através do serviço gerido *Confluent Cloud*. O mesmo nível de suporte aplica-se a qualquer outro provedor (como o *Amazon MSK* na AWS), blindando a empresa contra o *vendor lock-in*. Por outro lado, o RabbitMQ exigirá substituição na fase definitiva, pois os provedores de nuvem não oferecem o mesmo nível de serviço *PaaS* nativo para ele. Assim, ao optar pela agilidade do RabbitMQ no cenário Tático, a migração para o Azure Service Bus na Visão Target torna-se o caminho justificado e obrigatório para garantir o *Auto-scaling* e a gestão *FinOps*.

Alternativas Rejeitadas: A migração direta e imediata de toda a infraestrutura legada para a nuvem foi rejeitada por apresentar risco extremo de disrupção na frente de caixa e por atrasar a entrega de valor ao comerciante.

Restrições, Premissas e Experimentos:

- **Agnosticismo Tecnológico:** Como a escolha do provedor é uma decisão de gestão, estipulou-se a premissa de empacotar o "motor" da aplicação em contêineres universais. Isso evita o *vendor lock-in*, garantindo que o código possa ser migrado para AWS ou GCP no futuro.
- **Premissa Financeira (FinOps):** Reconhece-se que o modelo de nuvem (Visão Target) altera a estrutura de custos de capital fixo para custos operacionais variáveis, com uma estimativa mensal entre R$6.500,00 a R$10.500,00/mês. A aplicação das práticas de FinOps é uma premissa obrigatória para justificar este investimento. Através da visibilidade das métricas no Grafana/Log Analytics e da gestão de consumo *Pay-As-You-Go*, será possível otimizar os custos, escalando recursos apenas quando necessário (escala elástica). O custo superior é

justificado pelo ROI positivo gerado ao erradicar os custos de manutenção física, licenças de software legado e interrupções nas vendas.

Seção 4: Consequências

A aplicação das decisões arquiteturais propostas resultará no seguinte contexto operacional, introduzindo tanto benefícios claros quanto novos desafios sistêmicos:

Consequências Positivas:

- **Erradicação de Gargalos e Acoplamento:** A separação entre leitura e escrita (CQRS) garante que consultas complexas ou picos de acesso ao relatório gerencial nunca travem o registro de vendas no caixa do lojista.
- **Resiliência e Tolerância a Falhas:** A introdução de mensageria assíncrona criará um sistema onde quedas no serviço de consolidação não resultam em perda de dados de transações (offset de segurança).
- **Observabilidade Proativa:** A equipe de engenharia ganhará rastreabilidade ponta a ponta. Com o OpenTelemetry (por exemplo) gerando as métricas e o Grafana (por exemplo), centralizando a visualização, falhas silenciosas serão detectadas proativamente, e a dor de negócio de "acesso manual a servidores via RDP/SSH para ler arquivos de texto" será eliminada, otimizando o MTTR *(Mean Time to Resolution* – tradução: tempo médio de resolução).
- **Segurança de Nível Corporativo:** A Visão Target eleva a postura de segurança com a introdução do padrão Zero-Trust(confiança zero), utilizando isolamento de rede via Private Endpoints, WAF (Web Application Firewall) e identidades gerenciadas.
- **Controle e Otimização de Custos (FinOps):** A arquitetura habilitará a prática de **FinOps**, permitindo que as equipes de negócios e engenharia colaborem na otimização da infraestrutura. A capacidade de desligar instâncias ociosas (*Scale-to-Zero*) e acompanhar o faturamento por consumo garantirá previsibilidade orçamentária e proteção contra desperdícios (mitigando surpresas na fatura).

Consequências Negativas ou Desafios (Trade-offs):

- **Aumento Exponencial da Complexidade Técnica:** A equipe de desenvolvimento passará a lidar com sistemas distribuídos, eventuais problemas de consistência assíncrona e bancos de dados poliglotas.
- **Aumento da Complexidade de Governança (FinOps):** O modelo *Pay-As-You-Go* exige monitorização constante. Sem uma governança rigorosa de FinOps, o custo elástico da nuvem pode facilmente ultrapassar as estimativas mensais em casos de código mal otimizado ou ataques volumétricos não bloqueados pelo *Rate Limiting*.
- **Retrabalho Planejado:** A adoção da Visão Tática de transição exigirá que a equipe crie infraestrutura intermédia (como o Kafka local e instâncias do Grafana) que, posteriormente, serão descartadas e substituídas pelos serviços totalmente gerenciados do Azure ou por outro provedor em nuvem de escolha pela gestão da empresa.



Diante do registro de decisão de arquitetura, no qual apresentou as propostas arquiteturais para o negócio gestão financeira nas visões TÁTICA e TARGET, será contemplado o arquivo **Arquitetura-Solucao.drawio**, cuja proposta é expandir a visualização das visões em questão.